

An Efficient Compute-in-Memory based Accelerator for Point-based Point Cloud Neural Networks

Xipeng Lin, Cong Wang, Shanshi Huang, Hongwu Jiang

The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China

Email: {xlin297, cwang841}@connect.hkust-gz.edu.cn, {shanshihuang, hongwujiang}@hkust-gz.edu.cn

Abstract—Point-based point cloud neural networks (PNNs) have shown remarkable accuracy in various applications. However, the performance of PNNs is usually limited on resource-constrained edge devices. This paper presents Point-CIM, an efficient Compute-in-Memory (CIM) based accelerator for PNNs, with software and hardware co-optimization. A reconfigurable CIM unit, which exploits the inherent zero-bit-skipping capability of CIM array, is designed to efficiently process Multiply-Accumulate (MAC) operations on decomposed feature data with high bit sparsity. A Voxel-Morton-sorted Partitioning (VMP) method combined with Channel-wise Minimum (CM) base point selection is proposed to improve the decomposition bit sparsity further and reduce the hardware implementation overhead. Additionally, a detailed Bit-level Truncation Quantization (BTQ) method is proposed to directly compress the bit-width of the offset without incurring any additional hardware overhead. Based on this decomposition method, a Pre-decomposition (PD) data movement strategy is employed to reduce data transfers of intermediate features between the on-chip buffer and CIM array. Extensive evaluation experiments on multiple datasets show that Point-CIM achieves an average speedup of $1.69\times$ to $9.63\times$, and $3.11\times$ to $17.32\times$ improvement in energy efficiency, compared to state-of-the-art accelerators and general-purpose processors.

Index Terms—compute-in-memory, accelerator, point neural network, bit truncation

I. INTRODUCTION

3D point cloud data captured by LIDAR sensors or RGB cameras can accurately model the shape and surface characteristics of objects and environments. Unlike 2D pixels, 3D point cloud data is inherently unstructured and contains spatial positioning information, making its data processing and analysis complex and demanding. To address the challenges, Point-based PNNs, like PointNet [1] and PointNet++ [2], have been proposed to learn embedding features from raw points directly.

PointNet++, an extension of PointNet, specializes in capturing local structures and hierarchical features of 3D point clouds. Due to its promising performance, PointNet++ has become the foundational architecture for most Point-based PNNs [3]–[5]. Although Point-based PNNs show excellent accuracy, their heavy computational demands limit their deployment in resource-constrained edge devices. To tackle this problem, various software-hardware co-optimization methods have been proposed to improve the hardware efficiency. Most previous works [6]–[8] mainly focus on optimizing the farthest point sampling (FPS) and neighbor search stages, whereas the feature learning stage, typically executed by multi-layer perceptron (MLP), receives less attention. However, this stage

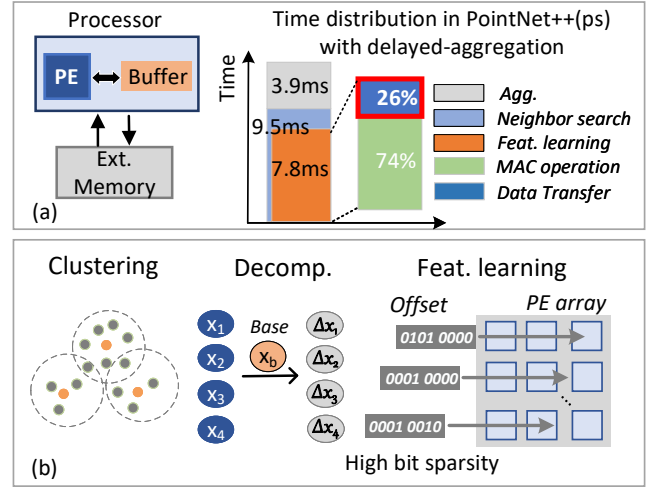


Fig. 1. (a) Time distribution in the point cloud processing task [9]; (b) overview of decomposition-based method.

takes more than half of the computation time of the PointNet++ executed on GPUs (Fig. 1(a)). Recently, several works revealed that the computation efficiency could be improved by exploiting the similarity in the features of points in closer spatial proximity [10]–[12]. As illustrated in Fig. 1(b), this insight allows for the decomposition of feature data into base and offset. The offset contains a higher proportion of zero-value bits and has a more concentrated distribution, enabling a further reduction in redundant computations of offset data. To fully convert this software optimization to real hardware performance improvement, customized processing units are designed to support zero-skipping operations and distribution statistics analysis for dynamic precision quantization on offsets [11], [12]. While previous works focus on the acceleration of MAC operations, the massive data movements also introduce substantial latency during the feature learning stage. As shown in Fig. 1(a), data transfer between the external memory and the processing units accounts for 26% of the processing time during the feature learning stage (tested on Nvidia Jetson TX2 [13]), which limits further improvement in the accelerator throughput.

Compute-in-memory (CIM) has emerged as a promising solution for data-centric applications to reduce the data movements [14]. By integrating the computation units into memory units and employing a weight stationary dataflow for MAC

operations, CIM avoids the data transfer of weight parameters, thus enhancing both the speed and energy efficiency of accelerators. Additionally, as the MAC operations in the CIM array are generally done in a digit-decomposed manner with binary inputs, zero-bits in the inputs are encoded as 0V and thus consume no energy [15]. In other words, CIM inherently supports zero-skipping operations without requiring any extra specialized designs in terms of energy consumption. Consequently, the energy efficiency of CIM-based accelerators is highly sensitive to the bit sparsity of inputs. On the other hand, the latency of the MAC operations are proportional to the bit-width of input data, which is directly affected by the quantization method. Furthermore, although the weight transfers are eliminated, there is still data movement of intermediate features on-chip. The latency and energy consumption caused by it remain significant overheads that cannot be overlooked [16].

To address the aforementioned problems and fully utilize the hardware advantages of CIM, we propose Point-CIM, an efficient CIM-based accelerator for PNNs. Software-hardware co-optimization techniques have been applied to improve the efficiency of Point-CIM. Our key contributions can be summarized as follows:

- We propose a reconfigurable CIM-based processing unit to support MAC operations on decomposed inputs with high bit sparsity. Two different modes are designed to process base and offset calculations, respectively. To further improve the hardware performance, a Pre-decomposition (PD) dataflow is employed to reduce data transfers of intermediate feature results on chip.
- We introduce a Voxel-Morton-sorted Partitioning (VMP) method combined with Channel-wise Minimum (CM) base point selection to ensure positive offset values and maintain high bit sparsity, which benefits the subsequent CIM computing scheme. Additionally, we introduce a detailed Bit-level Truncation Quantization (BTQ) method, designed to directly compress the bit-width of the offset without incurring any additional hardware overhead.
- Evaluated on various datasets, Point-CIM can achieve an average speedup ranging from $1.69\times$ to $9.63\times$, along with $3.11\times$ to $17.32\times$ improvement in energy efficiency, compared to state-of-the-art accelerators and general-purpose processors.

II. BACKGROUND

A. Point-based Point Cloud Neural Networks

Point-based PNNs typically consist of several Set Abstraction (SA) layers in serial. Each SA layer is composed of FPS, neighbor search, and feature learning stages. Firstly, FPS is applied to select a subset of points that represents the overall structure of the point cloud. Next, the neighbor search groups neighboring points around each sampled point. Finally, MLP layers are performed to extract the local feature within each group during the feature learning stage.

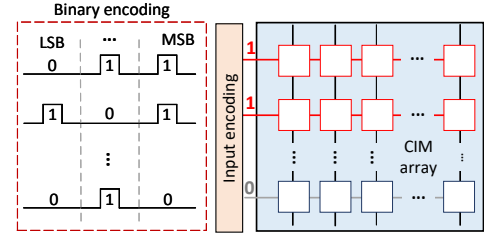


Fig. 2. The binary input encoding process for CIM.

Given an FPS-sampled point x_i and its neighboring group $\mathcal{N}(x_i)$, the final output of each SA layer is determined by

$$y_i = \max_{1 \leq j \leq J} \sigma(w \cdot (x_i^{(j)} - x_i))$$

where w represents the weights, $x_i^{(j)}$ denotes the j -th neighboring point in $\mathcal{N}(x_i)$, and $\sigma(\cdot)$ is the activation function (e.g. ReLU).

B. Feature Similarity in Close Spatial Proximity

Several previous works [10]–[12] have observed that points located near each other in space tend to share similar geometric features. This property can be leveraged to decompose features into two components: a base and an offset. For points within the same cluster, denoted as $x_c^k \in \mathcal{C}(x_c)$. They share a common base point x_c , with each point can be represented as:

$$x_c^k = x_c + (x_c^k - x_c) = x_c + \Delta x^k$$

where Δx^k represents the offset value of each point in this cluster. After this decomposition, the convolutional operation can be transformed into:

$$w \cdot x_c^k = w \cdot x_c + w \cdot \Delta x^k$$

which offers three advantages in theory. Firstly, since all points within the same cluster share a common base, the convolutional result of the base point $w \cdot x_c$ can be reused for all points of this cluster. Secondly, points that are close in space often show strong feature similarity, meaning that the offset Δx^k tends to have a more concentrated distribution. This allows Δx^k to be quantized with lower precision. Thirdly, the effective bits of each value are reduced greatly, which enables reduce redundant computations by skipping unnecessary zero-bit calculations.

C. CIM-based CNN accelerator

To tackle the memory wall bottleneck, researchers have introduced the in-memory computing paradigm, where computations are conducted directly within the memory array [17]–[19]. In CIM, the weight parameters are mapped into the crossbar memory array. As illustrated in Fig. 2, to perform MAC operations, the input vector is usually encoded into a binary sequence, with positive numbers represented in their original form and negative numbers in two's complement. Zero volts and power supply voltages are used to represent

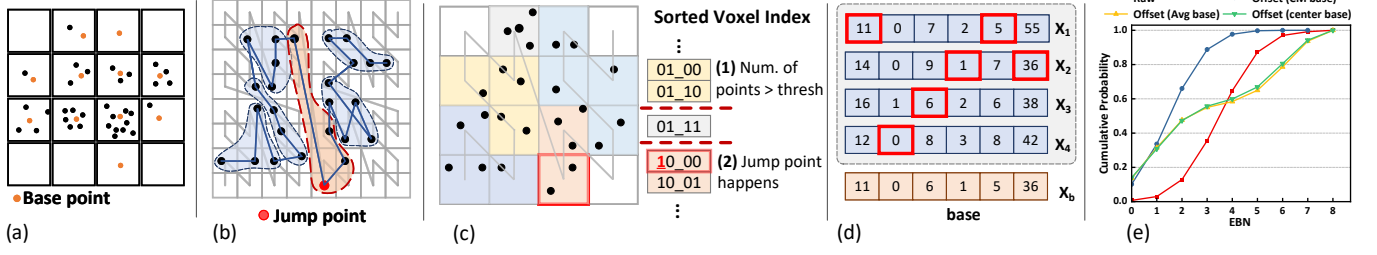


Fig. 3. (a) Uniform voxel partition method; (b) morton-sorted partition method; (c) Voxel-Morton-sorted Partition method (take a 2D space with Morton encoding using 2 MSBs as an example); (d) Channel-wise Minimum base point selection; (e) CDF of EBN for raw data and offsets in SA1 of PointNet++(m) with different base types.

logic “0” and “1”, respectively. The “0” input bits, in terms of 0V, do not contribute any current, consuming no energy cost. Accordingly, the CIM paradigm can naturally support skip zero bits of the input and the energy efficiency of the CIM-based accelerators is significantly affected by the bit sparsity of input series.

III. METHODOLOGY

This section discusses the software-hardware co-design techniques adopted for Point-CIM. In this section, all models’ weights and activations are originally quantized to 8 bits. Additionally, two definitions need to be clarified:

- **Effective Bit Number (EBN):** This refers to the number of bits equal to “1” in a fixed-point value, representing the sparsity of the value at the bit level. Lower EBN means higher bit-level sparsity.
- **Effective Bit Length (EBL):** This refers to the valid bit length of a fixed-point value from the highest non-zero bit to the least significant bit (LSB).

As an example, for an 8-bit fixed-point number 00101010₍₂₎, the EBN is 3, and the EBL is 6.

A. Partitioning Method and Base Point selection

Before decomposing the features, it is necessary to partition the 3D space into clusters of spatially proximate points. GDPCA [11] partitions the 3D space into uniformly distributed, fixed-size voxels. However, the number of points within each voxel may vary greatly, leading to significant workload imbalances, as shown in Fig. 3(a). To balance the size of the clusters, MoC [12] utilizes Morton code to encode and sort the coordinates of all points. All points are partitioned into uniform regions with fixed intervals. However, as shown in Fig. 3(b), the jump points usually reflect huge changes in spatial location, which will be clustered into wrong groups with uniform partitions. Additionally, for tasks with large input sizes, the significant number of points poses challenges for sorting circuits. For example, in the S3DIS dataset [20], each input contains 4096 points. To balance the computing efficiency and accuracy performance, a Voxel-Morton-sorted Partitioning (VMP) method is proposed in this work.

Voxel-Morton-sorted Partitioning (VMP) method. As shown in Fig. 3(c). Instead of encoding the full precision

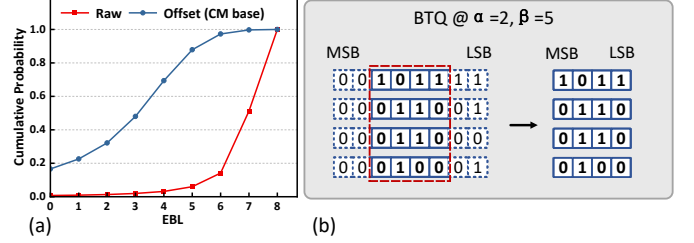


Fig. 4. (a) CDF of EBL for raw data and offsets in SA1 of PointNet++(m); (b) Bit-level Truncation Quantization method (Take $\alpha = 2$ and $\beta = 5$ for example).

of coordinates, we only encode the first 3 MSBs of each coordinate with Morton code (e.g. [101xxx, 001xxx, 010xxx] is encoded as 001_100_011). In this way, the 3D space with xyz dimension will be partitioned into uniform 512 ($8 \times 8 \times 8$) voxels with index encoded with 9 bits. Consequently, we only need to sort 512 blocks, rather than all points. The partitioning is conducted based on two principles: (1) the total number of each partition’s points exceeds the workload threshold; (2) an index jump occurs. In our setting, the workload threshold is defined as the ratio of the number of points to the number of partitions. The jump point is identified when the 3rd MSB of the voxel index flips, indicating a location jump in space. As a result, points from different voxels around jump point will not be encoded to the same partition. Inside each voxel, the points are grouped by storage ordering with loose locality constraints.

Channel-wise Minimum (CM) Base Point. The base point of the decomposition can be decided in different ways. GDPCA [11] takes the point nearest to the center of each voxel (Center base), while MoC [12] takes the average of features in each group (Avg base). The former suffers from the overhead of distance calculation, while the latter may lead to nearly symmetric offsets around zero. For performing CIM operations, the inputs are encoded in 2’s complement format. Thus, near-zero negative values for offsets (e.g. -1, -2) in this format contain lots of “1” bits (yellow line in Fig. 3(e)), rapidly reducing the bit sparsity of offset data and increasing the energy cost of in-memory computing. To address this issue, the base point is defined as a vector of channel-wise minimum feature values across different data points within the same partition, ensuring that all offsets are positive (Fig.

function are sent back to the on-chip buffer for temporary storage. As the chip size scales up, this data movement could lead to significant hardware overhead. To mitigate this, we propose a Pre-decomposition (PD) Dataflow, which shifts the decomposition and BTQ operation from the input of a certain layer to the output of the previous layer. As shown in Fig. 5(c), the network for feature learning consists of multiple layers of perceptrons, with the last layer followed by aggregation and max-pooling operations. In the original dataflow, the decomposition and BTQ are applied on the inputs of each layer and then fed into the MAC units for further processing. The raw outputs of each layer are sent back to the buffer for temporal storage and used as the inputs of the next layer later. In this dataflow, the raw data transmitted are in full precision. In the PD dataflow, the intermediate data between perceptron layers inside the same block are decomposed and BTQed before being sent back to the buffer. In this way, the bases are transferred in full precision and offsets in reduced precision, enabling reducing total transmitted bits. Consequently, the overhead in data transfer between buffer and CIM array can be mitigated significantly. For the last layer in the SA, as the raw data has to be applied to the Agg unit and Maxpool unit, the decomposition and BTQ for the next layer could not be applied in advance.

V. EVALUATION

A. Experimental Setup

Algorithmic Setup. To evaluate our design, we select the representative Point-based model, PointNet++, and assess its performance across three datasets, as shown in Table I. All models are implemented using PyTorch and trained to reach state-of-the-art performance. Subsequently, the models were retrained using the delayed-aggregation method [9]. Finally, all models were quantized to 8-bit weights and 8-bit activations. The accuracy performance of all models is compared in Table II.

Hardware Setup. For the feature learning stage, the hardware performance for MLP-related computations is evaluated using the Neurosim framework [16], [21]. NeuroSim is an open-source simulator that can build the CIM architecture with traced inputs from real networks to evaluate the overall hardware performance. Other components, such as the Mapping Unit and Agg Unit, are directly adopted from GDPCA [11].

Baseline. To evaluate the accuracy impact and hardware performance improvement of Point-CIM, we choose the 8-bit quantized model as the accuracy baseline. Additionally, we adopt three types of platforms as hardware baselines, including CPU (Intel Xeon Gold 6226R), GPU (Tesla A100), and state-of-the-art ASICs (Mesorasi [9], PointAcc [22], GDPCA [11]).

B. Parameters Setting

The hardware design parameters are presented in Table III. Point-CIM is compared with several prior customized accelerators, which are all based on 40 nm technology node and operate at a frequency of 1 GHz. Point-CIM is implemented

TABLE I
EVALUATION BENCHMARKS

Application	Dataset	Model	Notation
Classification	ModelNet40 [23]	PointNet++ (SSG)	PointNet++(m)
Part Segmentation	ShapeNet [24]		PointNet++(ps)
Segmentation	S3DIS [20]		PointNet++(s)

TABLE II
MODEL ACCURACY (%)

Dataset	Original Model	Delayed-Aggregation	w:8,a:8
ModelNet40	92.2	91.3	91.3
ShapeNet	85.2	84.6	84.4
S3DIS	54.3	54.6	53.7

*w:8 and a:8 mean the weight and activation are quantized with 8-bit precision.

TABLE III
HARDWARE CONFIGURATIONS

Chip	Mesorasi [9]	PointAcc [22]	GDPCA [11]	Point-CIM
Tech(nm)	40	40	40	45
Frequency(MHz)	1000	1000	1000	1000
Buffer(KB)	2048	776	2560	2048
Bandwidth (GB/s)	HBM	HBM	HBM	HBM
MACs	64 × 64	64 × 64	64 × 64	SRAM:4.4MB
	Systolic Array	Systolic Array	BSP Array	CIM Array

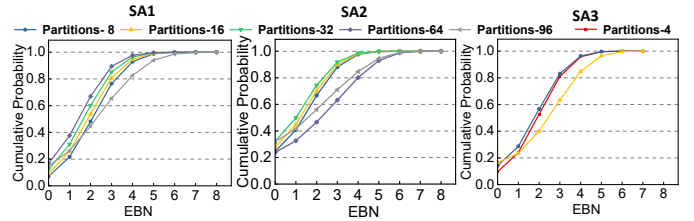


Fig. 6. CDF of EBN for offsets in SA1, SA2, and SA3 of PointNet++(m).

with 45 nm technode and operates at 1 GHz. Mesorasi and PointAcc utilize 64×64 systolic array MACs, while GDPCA introduces 64×64 BSP array MACs. Point-CIM employs an SRAM-based CIM architecture. The computational capability of the CIM architecture is evaluated based on the capacity of the SRAM, which should be able to store all weight parameters of models. In our experiment, we configured the CIM-SRAM with a capacity of 4.4 MB. All architectures utilize HBM memory with a bandwidth of 256 GB/s.

The number of partitions is a critical parameter that needs to be specified before experiments. In this section, we present the parameter setting for PointNet++(m) as an example. The same analytical process can be applied to other benchmarks to determine the optimal number of partitions. As shown in Fig. 6, when the number of partitions for SA1 is set to 64, EBN achieves the sparsest distribution. The optimal number of partitions for SA2 is 32; while for SA3, it is 8. These parameter settings align with the characteristics of PointNet++. As the depth increases, the number of points in each layer decreases, requiring fewer partitions.

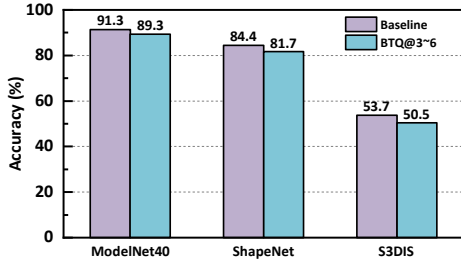


Fig. 7. The accuracy comparison between baseline models and models quantized with BTQ.

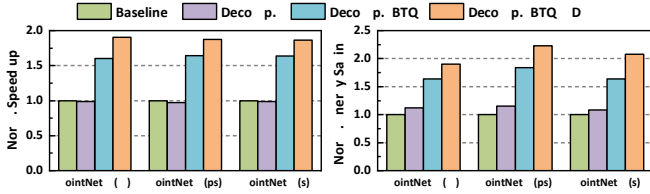


Fig. 8. The speedup and energy savings achieved by BTQ and PD dataflow are compared to the baseline model.

The configurable parameters α and β are set to be 3 and 6 according to case study, which means the 3rd to 6th bits of offsets are reserved for CIM operations.

C. Accuracy Analysis

To analyze the the impact of BTQ on inference accuracy of models, we compare the the accuracy between models quantized with BTQ and the baseline across three different datasets. As shown in Fig. 7, after BTQ, the accuracies decrease by 2.0%, 2.7%, and 3.2% across three datasets, respectively. The decreases in accuracy are within an acceptable range for our experiments.

D. Hardware Performance Analysis

Fig. 8 compares the speedup and energy savings in feature learning stage optimized with BTQ and PD dataflow against the baseline across three different models. After decomposition, Point-CIM achieves increases in energy efficiency by 1.12 $\times$, 1.15 $\times$, and 1.08 $\times$, respectively. This indicates that the offset values show much higher bit sparsity after decomposition and the CIM-based computing unit can handle inputs with high bit sparsity with higher energy efficiency. Furthermore, after applying BTQ, Point-CIM achieves improvements in energy efficiency by 1.64 $\times$, 1.84 $\times$, and 1.64 $\times$, respectively. Meanwhile, Point-CIM demonstrates speed-up enhancement of 1.61 $\times$, 1.64 $\times$, and 1.64 $\times$, respectively. At last, when both BTQ and PD dataflow are applied, energy savings increase by factors of 1.90 $\times$, 2.23 $\times$, and 2.07 $\times$, while speed-up is improved by 1.91 $\times$, 1.87 $\times$, and 1.86 $\times$, respectively. It suggests the effectiveness of PD dataflow in mitigating the transfer overhead of intermediate features.

Fig. 9 provides a comparative analysis of the normalized speedup and energy efficiency of Point-CIM relative to other hardware baseline on PointNet++(m). Specifically, Point-CIM

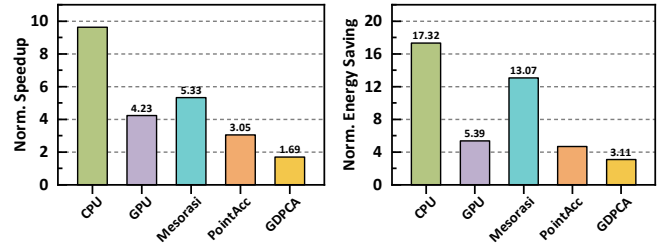


Fig. 9. The speedup and energy saving of Point-CIM over other platforms

achieves an average speedup of 9.63 $\times$, 4.23 $\times$, 5.33 $\times$, 3.05 $\times$, and 1.69 $\times$ over CPU, GPU, Mesorasi, PointAcc, and GDRCA, respectively. Moreover, Point-CIM shows notable improvements in energy efficiency, offering 17.32 $\times$, 5.39 $\times$, 13.07 $\times$, 4.67 $\times$, and 3.11 $\times$ higher energy savings compared to other platforms. These results highlight Point-CIM's superior performance in both computational speed and energy efficiency when compared to other hardware platforms.

VI. CONCLUSION

This paper presents Point-CIM, an efficient CIM-based accelerator for PNNs, with software and hardware co-optimization. A reconfigurable CIM unit is designed to efficiently process MAC operations on decomposed feature data with high bit sparsity. A novel VMP method combined with base point selection method is proposed to further improve the offset bit sparsity and reduce the hardware overhead. Additionally, a BTQ method is proposed to directly compress the bit-width of the offset without incurring any additional hardware overhead. To reduce data transfers of intermediate features between the on-chip buffer and CIM array, a PD data movement strategy is employed to optimize dataflow. Extensive experiments show that Point-CIM achieves an average speedup of 1.69 $\times$ to 9.63 $\times$, and 3.11 $\times$ to 17.32 $\times$ improvement in energy efficiency, compared to state-of-the-art accelerators and general-purpose processors.

VII. ACKNOWLEDGMENT

This work is supported in part by the National Natural Science Foundation of China (Grant No. 62304195), Guangzhou-HKUST(GZ) Joint Funding Program (No. 2024A03J0540), Guangzhou-HKUST(GZ) Joint Funding Program (No. 2024A03J0541), Guangzhou Municipal Science and Technology Project (Municipal Key Laboratory Construction Project, Grant No.2023A03J0013)

REFERENCES

- [1] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.
- [2] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," *Advances in neural information processing systems*, vol. 30, 2017.
- [3] A. V. Phan, M. Le Nguyen, Y. L. H. Nguyen, and L. T. Bui, "Dgcn: A convolutional neural network over large-scale labeled graphs," *Neural Networks*, vol. 108, pp. 533–543, 2018.

- [4] H. Zhao, L. Jiang, J. Jia, P. H. Torr, and V. Koltun, "Point transformer," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 16259–16268.
- [5] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, "Frustum pointnets for 3d object detection from rgb-d data," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 918–927.
- [6] M. Han, L. Wang, L. Xiao, H. Zhang, C. Zhang, X. Xu, and J. Zhu, "Quickfps: Architecture and algorithm co-design for farthest point sampling in large-scale point clouds," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 11, pp. 4011–4024, 2023.
- [7] X. Yang, T. Fu, G. Dai, S. Zeng, K. Zhong, K. Hong, and Y. Wang, "An efficient accelerator for point-based and voxel-based point cloud neural networks," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [8] Y. Lian, X. Yang, K. Hong, Y. Wang, G. Dai, and N. Xu, "A point transformer accelerator with fine-grained pipelines and distribution-aware dynamic fps," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [9] Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu, "Mesorasi: Architecture support for point cloud analytics via delayed-aggregation," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1037–1050.
- [10] Z. Ying, S. Zhao, S. Bhuyan, C. S. Mishra, M. T. Kandemir, and C. R. Das, "Pushing point cloud compression to the edge," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 282–299.
- [11] C. Chen, X. Zou, H. Shao, Y. Li, and K. Li, "Point cloud acceleration by exploiting geometric similarity," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1135–1147.
- [12] X. Liu, Z. Song, H. Chen, X. Li, and X. Liang, "Moc: A morton-code-based fine-grained quantization for accelerating point cloud neural networks," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [13] D. Franklin, "Nvidia jetson tx2 delivers twice the intelligence to the edge," *NVIDIA Accelerated Computing—Parallel Forall*, vol. 6, 2017.
- [14] J.-s. Seo, J. Saikia, J. Meng, W. He, H.-s. Suh, Y. Liao, A. Hasssan, I. Yeo *et al.*, "Digital versus analog artificial intelligence accelerators: Advances, trends, and emerging designs," *IEEE Solid-State Circuits Magazine*, vol. 14, no. 3, pp. 65–79, 2022.
- [15] S. Huang, H. Jiang, and S. Yu, "Hardware-aware quantization/mapping strategies for compute-in-memory accelerators," *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 3, pp. 1–23, 2023.
- [16] X. Peng, S. Huang, H. Jiang, A. Lu, and S. Yu, "Dnn+ neurosim v2. 0: An end-to-end benchmarking framework for compute-in-memory accelerators for on-chip training," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 11, pp. 2306–2319, 2020.
- [17] L. Song, X. Qian, H. Li, and Y. Chen, "Pipelayer: A pipelined rram-based accelerator for deep learning," in *2017 IEEE international symposium on high performance computer architecture (HPCA)*. IEEE, 2017, pp. 541–552.
- [18] T. Yang, D. Li, Y. Han, Y. Zhao, F. Liu, X. Liang, Z. He, and L. Jiang, "Pimgen: A rram-based pim design for graph convolutional network acceleration," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 583–588.
- [19] W. Li, X. Sun, S. Huang, H. Jiang, and S. Yu, "A 40-nm mlc-rram compute-in-memory macro with sparsity control, on-chip write-verify, and temperature-independent adc references," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 9, pp. 2868–2877, 2022.
- [20] I. Armeni, O. Sener, A. R. Zamir, H. Jiang, I. Brilakis, M. Fischer, and S. Savarese, "3d semantic parsing of large-scale indoor spaces," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 1534–1543.
- [21] C. Wang, Z. Chen, and S. Huang, "Micsim: A modular simulator for mixed-signal compute-in-memory based ai accelerator," *arXiv preprint arXiv:2409.14838*, 2024.
- [22] Y. Lin, Z. Zhang, H. Tang, H. Wang, and S. Han, "Pointacc: Efficient point cloud accelerator," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 449–461.
- [23] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3d shapenets: A deep representation for volumetric shapes," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1912–1920.
- [24] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su *et al.*, "Shapenet: An information-rich 3d model repository," *arXiv preprint arXiv:1512.03012*, 2015.